

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1104

COURSE OUTCOME COVERED

***CO1:** Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr.Gayathri.A

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

*I **R.Durga Sree/192411189**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1. Suppose a software system is developed for a **Smart Banking System** where customers can open accounts, transfer funds, and pay utility bills.

(a) Explain the following object-oriented concepts:

- Class
- Object
- Message Passing
- Encapsulation
- Polymorphism

(b) Describe the following object relationships:

- Association
- Aggregation
- Composition

(c) Explain how these concepts improve software reusability and maintainability in a banking system.

(a) Explain the following object-oriented concepts

1. Class

- A class is a blueprint or template used to create objects.
- It defines the data (attributes) and functions (methods) of an object.
- **Example:** Customer, BankAccount, and Transaction classes.

2. Object

- An object is an instance of a class.
- It contains actual values and performs operations defined by the class.
- **Example:** A customer named "Rahul" having Savings Account No. 12345.

3. Message Passing

- Message passing is the communication between objects by invoking methods.
- It allows objects to work together without exposing internal details.
- **Example:** A Customer object sends a request to the BankAccount object to transfer funds.

4. Encapsulation

- Encapsulation combines data and methods into one unit.
- Data is protected using private access, and it can only be accessed through public methods.
- **Example:** Account balance is private and updated only through deposit() and withdraw() methods.

5. Polymorphism

- Polymorphism allows one interface to perform different actions.
- The same method behaves differently depending on the object.
- **Example:** calculateInterest() works differently for Savings and Current accounts.

(b) Object Relationships

1. Association

A relationship where two objects interact but are independent.
Both objects can exist separately.

Example: Customer uses BankAccount.

2. Aggregation

A weak "has-a" relationship.
The child object can exist even if the parent is removed.

Example: Bank has many Customers.

3. Composition

A strong "has-a" relationship.
The child object cannot exist without the parent.

Example: BankAccount contains Transactions. Deleting the account removes its transactions.

(c) How these concepts improve software reusability and maintainability

- ❖ Reusable classes reduce duplicate code.
- ❖ Encapsulation protects sensitive banking information.
- ❖ Polymorphism makes it easy to add new account types.
- ❖ Relationships organize system structure.
- ❖ Maintenance becomes easier because changes affect only related classes.

2. Consider developing a **Smart Healthcare Management System**.

(a) Explain the following object-oriented design principles:

- Abstraction
- Encapsulation
- Modularity

(b) Describe the following SOLID principles:

- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)

(c) Explain how these principles improve software quality and flexibility.

(a) Object-Oriented Design Principles

1. Abstraction

- Shows only essential information while hiding implementation details.
- Reduces system complexity.
- Example: Doctors use patient records without knowing database implementation.

2. Encapsulation

Protects patient information using private data.

Access is provided through public methods.

3. Modularity

- Divides the system into independent modules.
- Each module performs a specific function.
- Example: Appointment, Billing, Pharmacy, and Patient modules.

(b) SOLID Principles.

1. Liskov Substitution Principle (LSP)

- A subclass should replace its parent class without affecting program behavior.
- Example: SpecialistDoctor can replace Doctor wherever required.

2. Interface Segregation Principle (ISP)

- Clients should not depend on interfaces they do not use.
- Create small, specific interfaces instead of one large interface.
- **Example:** Billing staff use Billing interface, while Doctors use Medical interface.

(c) Benefits

- Improves code quality.
 - Reduces dependency between modules.
 - Makes software flexible and scalable.
 - Simplifies testing and maintenance.
 - Supports future enhancements.
-

3. Suppose a software company is developing an **Online Food Delivery System**.

(a) Explain the phases of the Software Development Life Cycle (SDLC):

- Planning
- Requirement Analysis
- Design
- Development
- Testing
- Maintenance

(b) Describe the purpose of each SDLC phase.

(c) Explain how SDLC contributes to delivering reliable software.

(a) Explain the phases of the Software Development Life Cycle (SDLC)

1. Planning

- Identifies project goals and objectives.
- Estimates budget, resources, and project timeline.
- Checks the feasibility of the project.

2. Requirement Analysis

- Collects customer and user requirements.
- Analyzes and documents requirements in the SRS.
- Defines the system functionalities.

3. Design

- Creates the software architecture.
- Designs the database and user interface.

- Prepares UML diagrams and system design.

4. Development

- Developers write the program code.
- Implements all required features.
- Integrates different modules of the system.

5. Testing

- Identifies and fixes software defects.
- Verifies that all features work correctly.
- Ensures the software meets user requirements.

6. Maintenance

- Fixes bugs after deployment.
- Improves software performance.
- Adds new features and updates.

(b) Describe the purpose of each SDLC phase

Planning

- Defines the project scope.
- Estimates cost and schedule.

Requirement Analysis

- Understands customer needs.
- Documents all requirements.

Design

- Provides a blueprint for development.
- Defines system structure.

Development

- Converts the design into working software.
- Implements all functionalities.

Testing

- Detects and removes errors.

- Ensures software quality.

Maintenance

- Supports software after deployment.
- Improves and updates the system.

(c) Explain how SDLC contributes to delivering reliable software

- Provides a systematic development process.
 - Ensures customer requirements are fulfilled.
 - Detects and fixes errors early.
 - Improves software quality and reliability.
 - Reduces development time and cost.
 - Makes the software easy to maintain and enhance.
 - Increases customer satisfaction.
-

4. Consider a project for developing a **Smart Agriculture Monitoring System**, where customer requirements change regularly.

(a) Explain the following SDLC models:

- Prototype Model
- Spiral Model
- Agile Model
- DevOps Model

(b) Compare the Prototype Model and Agile Model.

(c) Recommend the most suitable SDLC model for this project with justification.

(a) Explain the following SDLC models

1. Prototype Model

- A working prototype of the software is developed first.
- Users provide feedback on the prototype.
- Requirements are refined before developing the final system.
- Suitable when requirements are not clear.

2. Spiral Model

- Development is carried out in multiple cycles (spirals).
- Each cycle includes planning, risk analysis, development, and testing.

- Focuses on identifying and reducing risks.
- Suitable for large and high-risk projects.

3. Agile Model

- Software is developed in small iterations called sprints.
- Customer feedback is collected after each sprint.
- Changes can be made at any stage.
- Suitable for projects with frequently changing requirements.

4. DevOps Model

- Combines software development and operations.
- Supports Continuous Integration (CI) and Continuous Deployment (CD).
- Automates testing, deployment, and monitoring.
- Delivers software faster with improved quality.

(b) Compare the Prototype Model and Agile Model

Prototype Model

- Develops a sample model before the final product.
- Mainly focuses on understanding user requirements.
- User feedback is collected during the prototype stage.
- Best suited when requirements are unclear.

Agile Model

- Develops the software in small, working increments.
- Focuses on continuous development and delivery.
- Customer feedback is collected after every sprint.
- Best suited when requirements change frequently.

(c) Recommend the most suitable SDLC model with justification

- Recommended Model: Agile Model
 - Supports frequent changes in customer requirements.
 - Provides continuous customer feedback.
 - Delivers working software in short iterations.
 - Makes it easy to add or modify features.
 - Improves software quality through continuous testing.
 - Ensures faster delivery and higher customer satisfaction.
-

5. Suppose a software analyst is designing a **Smart Transport Management System**.

(a) Explain the importance of UML in object-oriented software development.

(b) Describe the following UML diagrams:

- State Diagram
- Component Diagram
- Deployment Diagram
- Package Diagram

(c) Explain how UML models improve communication among stakeholders.

(a) Explain the importance of UML in object-oriented software development

- UML (Unified Modeling Language) is a standard visual language used to design software systems.
- It helps developers understand the structure and behavior of the system.
- It provides a clear blueprint before coding begins.
- It improves communication among developers, analysts, and customers.
- It helps identify design errors at an early stage.
- It simplifies documentation and maintenance of the software.

(b) Describe the following UML diagrams

1. State Diagram

- Shows the different states of an object during its lifecycle.
- Describes how an object changes from one state to another based on events.
- Useful for modeling dynamic behavior.

2. Component Diagram

- Shows the different software components of a system.
- Describes how components interact with each other.
- Helps organize large software applications.

3. Deployment Diagram

- Shows the physical deployment of software on hardware devices.
- Represents servers, databases, and network connections.
- Helps understand the system's physical architecture.

4. Package Diagram

- Groups related classes into packages.

- Shows dependencies between different packages.
- Improves organization and maintainability of large projects.

(c) Explain how UML models improve communication among stakeholders

- Provides a common visual language for all stakeholders.
 - Makes complex system designs easier to understand.
 - Reduces misunderstandings between developers and customers.
 - Helps developers, testers, and analysts work together effectively.
 - Improves documentation and project planning.
 - Ensures everyone has a clear understanding of the system before development begins.
-

6. Consider developing a **Smart Inventory Management System**.

(a) Explain the concept of modular software design.

(b) Compare the procedural programming approach with the object-oriented programming approach.

(c) Explain how modularity improves software scalability, maintainability, and reusability.

(a) Explain the concept of modular software design

- Modular software design divides a software system into smaller, independent modules.
- Each module performs a specific function.
- Modules can be developed, tested, and maintained separately.
- Changes in one module have minimal impact on other modules.
- It improves code organization and software quality.

(b) Compare the procedural programming approach with the object-oriented programming approach

Procedural Programming

- Focuses on functions and procedures.
- Data and functions are separate.
- Code reusability is limited.
- Difficult to maintain large applications.
- Provides less data security.

Object-Oriented Programming (OOP)

- Focuses on objects and classes.
- Data and methods are combined into a single unit.
- Supports code reusability through inheritance and polymorphism.
- Easier to maintain and extend.
- Provides better data security through encapsulation.

(c) Explain how modularity improves software scalability, maintainability, and reusability

Scalability

- New modules can be added without affecting existing modules.
- Supports future expansion of the software.

Maintainability

- Bugs can be identified and fixed within individual modules.
- Updates and modifications are easier to perform.

Reusability

- Existing modules can be reused in other projects.
- Reduces development time and overall cost.

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps

